

Instituto Federal de Santa Catarina
Campus São José

Curso de Engenharia de Telecomunicações

Prática 2 – Gestão de Tarefas

Questionário de Entrega

Aluno: Victor E. L. Guerra

Professor: Rogério Pereira Junior

São José, 27/03/2026

Questionário de Entrega

1. Qual a diferença entre os estados Ready (Pronto) e Running (Executando) observada durante o uso do comando top?

No estado **Ready (Pronto)**, o processo está carregado na memória e apto a executar, porém aguarda que o escalonador lhe atribua a CPU. No estado **Running (Executando)**, o processo está efetivamente utilizando a CPU naquele instante.

No **top**, ambos os estados aparecem com o código R, pois o Linux agrupa processos prontos e em execução sob a mesma classificação. A alternância constante na ordem dos processos reflete as trocas de contexto realizadas pelo escalonador: a cada fatia de tempo, um processo diferente pode estar de fato na CPU, enquanto os demais aguardam na fila de prontos.

2. Ao executar pstree, você notou que o xeyes é filho de qual processo?

O **xeyes** é filho do processo **bash**, que é o shell a partir do qual o comando foi executado. A hierarquia típica observada é:

```
systemd -> terminal -> bash -> xeyes
```

Isso ocorre porque, no Linux, todo processo é criado por outro processo (seu pai) através da chamada de sistema **fork()**, formando uma árvore hierárquica.

3. Explique por que, ao aumentarmos o valor de nice para 15, o processo tende a usar menos CPU quando há outros processos ativos.

O escalonador CFS (*Completely Fair Scheduler*) do Linux utiliza o valor de *nice* para calcular o peso relativo de cada processo na distribuição de tempo de CPU. O valor de *nice* varia de -20 (maior prioridade) a $+19$ (menor prioridade), sendo 0 o padrão.

Com *nice* = 15, o processo recebe um peso significativamente menor em comparação com processos de *nice* = 0. Quando há outros processos ativos competindo pela CPU, o escalonador atribui fatias de tempo proporcionais ao peso de cada processo, fazendo com que o processo com *nice* 15 receba consideravelmente menos tempo de CPU.

Vale notar que, em um sistema ocioso (sem concorrência), mesmo um processo com *nice* alto pode utilizar toda a CPU disponível, pois não há outros processos disputando o recurso.

4. O que acontece com o registro de um processo no SO quando enviamos um sinal kill -9?

O sinal SIGKILL (9) é tratado diretamente pelo kernel e não pode ser interceptado, bloqueado ou ignorado pelo processo. Ao recebê-lo, o kernel encerra o processo imediatamente, sem permitir a execução de *handlers* de limpeza. Em seguida, o sistema operacional:

1. Libera todos os recursos alocados ao processo (memória, arquivos abertos, conexões de rede);

2. Notifica o processo pai através do sinal `SIGCHLD`;
3. Remove a entrada do processo na tabela de processos (PCB).

Se o processo pai não executar a chamada `wait()` para recolher o código de saída do filho, o processo encerrado permanece temporariamente no estado *zumbi* (Z) até ser recolhido.